

# Corporate Talent & Payroll Management System

## PYTHON CAPSTONE PROJECT SPECIFICATION

Welcome to your first real-world Python portfolio project! In this project, you will build a structural, modular program simulating a corporate HR and Payroll backend system. This project is fully suitable to showcase on your **GitHub** profile and add to your **CV** as a beginner software engineer.

🎯 **Project Goal:** Practice structural programming by breaking down business logic into standalone, reusable Python functions using clean code guidelines, parameters, and return statements.

### 1. Architecture & Required Functions

Your program must be strictly modular. You are required to design and implement the following standalone functions:

#### Function 1: `calculate_bonus(base_salary, performance_rating)`

- **Inputs:** `base_salary` (float), `performance_rating` (int from 1 to 5).
- **Logic:**
  - Rating 5: 20% bonus of the base salary (Excellent performance).
  - Rating 3 or 4: 10% bonus of the base salary (Good performance).
  - Rating below 3: 0% bonus (Needs improvement).
- **Output:** Returns the calculated `bonus_amount`.

#### Function 2: `calculate_tax(gross_salary)`

- **Inputs:** `gross_salary` (float) which equals `base_salary + bonus`.
- **Logic (Nested Conditions):**
  - If gross salary is greater than 7000: 15% tax deduction.
  - If gross salary is between 3000 and 7000 (inclusive): 10% tax deduction.
  - If gross salary is below 3000: 0% tax (Tax exempt).
- **Output:** Returns the calculated `tax_amount`.

#### Function 3: `main_hr_app()`

This is the controller function that orchestrates the entire runtime flow. It must perform the following actions:

1. Prompt the user to input the Employee's Name, Department, Base Salary, and Performance Rating.
2. Perform data validation (e.g., ensure rating is between 1 and 5, salary cannot be negative).

3. Call `calculate_bonus` and `calculate_tax` sequentially, passing data correctly via arguments.
4. Format and display a beautiful, readable textual profile summary summarizing the payroll breakdown.

## 2. GitHub & CV Best Practices

- **Input Validation:** Always handle unexpected inputs gracefully to demonstrate robust coding practices.
- **Variable Naming:** Use descriptive lowercase names with underscores (snake\_case) such as `net_salary` rather than ambiguous names like `x` or `s`.
- **Documentation:** Upload your code into a file named `main.py` and write a clean `README.md` file outlining the project description, installation instructions, and an example run.

## 3. Reference Implementation Example

Here is a basic blueprint demonstrating how your project should flow and look structurally:

```
# 1. Function to calculate performance bonus
def calculate_bonus(base_salary, performance_rating):
    if performance_rating == 5:
        bonus_percentage = 0.20
    elif performance_rating >= 3:
        bonus_percentage = 0.10
    else:
        bonus_percentage = 0.0

    return base_salary * bonus_percentage

# 2. Function to calculate progressive tax deductions
def calculate_tax(gross_salary):
    if gross_salary > 7000:
        tax_percentage = 0.15
    elif gross_salary >= 3000:
        tax_percentage = 0.10
    else:
        tax_percentage = 0.0

    return gross_salary * tax_percentage

# 3. Core runtime application
def main_hr_app():
    print("--- 🏢 Corporate Payroll System 🏢 ---")

    # User Inputs
    emp_name = input("Enter Employee Name: ")
    base_salary = float(input("Enter Base Salary (EGP): "))
    rating = int(input("Enter Performance Rating (1-5): "))

    # Input Validation Bonus
```

```
if rating < 1 or rating > 5 or base_salary < 0:
    print("❌ Invalid data entered. Please restart and check your inputs.")
    return
```

```
# Process Flow via Functions
bonus = calculate_bonus(base_salary, rating)
gross_salary = base_salary + bonus
tax = calculate_tax(gross_salary)
net_salary = gross_salary - tax
```

```
# Output Statement Generator
print("\n" + "="*40)
print(f"📄 PAYROLL STATEMENT FOR: {emp_name}")
print("="*40)
print(f"• Base Salary:           {base_salary:.2f} EGP")
print(f"• Earned Bonus:           {bonus:.2f} EGP")
print(f"• Tax Deductions:         {tax:.2f} EGP")
print("-" * 40)
print(f"💰 NET PAYABLE CASH: {net_salary:.2f} EGP")
print("="*40)
```

```
# Trigger Program Run
main_hr_app()
```